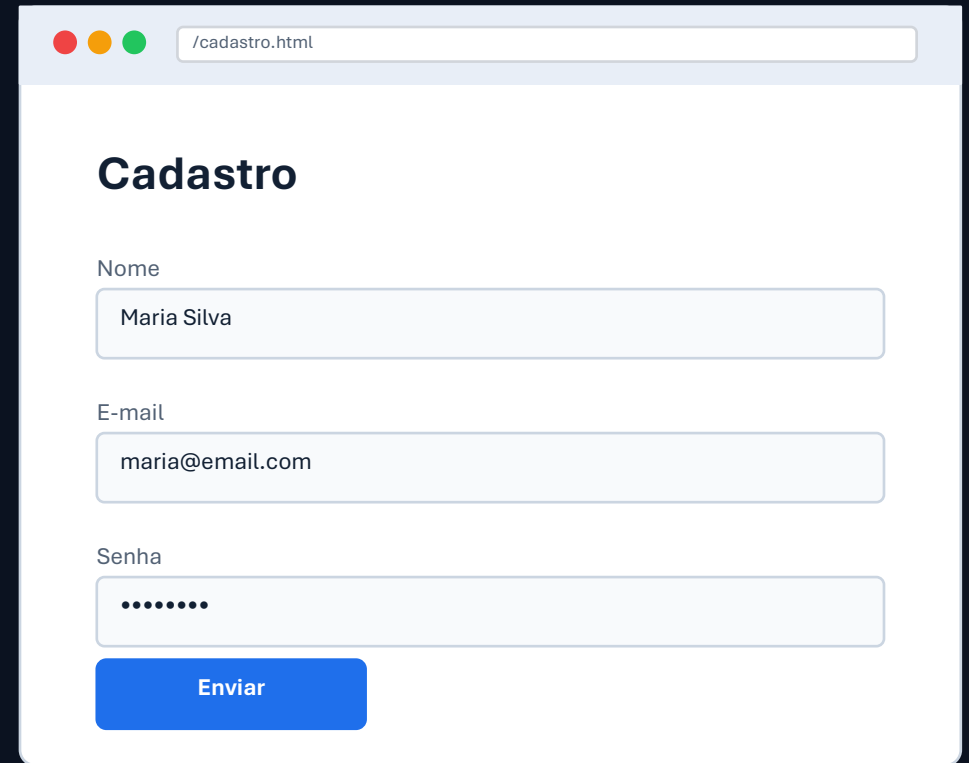


Programação para internet

Formulários, validação, acessibilidade e CSS

HTML dá significado ao conteúdo. CSS organiza a apresentação visual. Acessibilidade garante que mais pessoas consigam usar a página com autonomia.



The image shows a web browser window with the address bar displaying "/cadastro.html". The page content is a registration form titled "Cadastro". It contains three input fields: "Nome" (filled with "Maria Silva"), "E-mail" (filled with "maria@email.com"), and "Senha" (filled with masked characters "••••••"). Below the fields is a blue button labeled "Enviar".

Objetivos da aula

Ao final da aula, o aluno deverá compreender como construir formulários HTML, escolher tipos de campos adequados, aplicar validações básicas e reconhecer cuidados de acessibilidade. Também deverá entender como o CSS separa conteúdo de apresentação, usando seletores, cascata, herança, modelo de caixas e tipografia.



HTML + CSS + Acessibilidade

HTML, CSS e comportamento

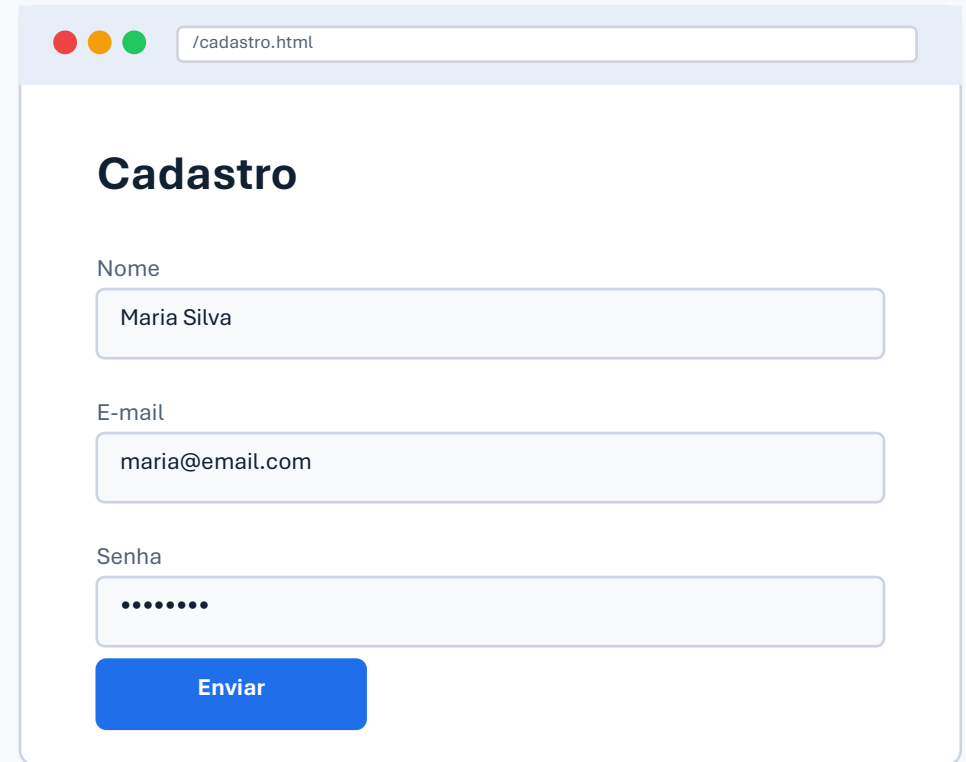
Em uma página web, o HTML descreve a estrutura e o significado do conteúdo, enquanto o CSS define como esse conteúdo será apresentado visualmente. JavaScript pode acrescentar comportamento, mas bons formulários e estilos começam com marcação semântica, nomes claros, rótulos corretos e regras CSS reutilizáveis.



HTML + CSS + Acessibilidade

Formulários como ponto de interação

Formulários são áreas da página em que o usuário informa dados, escolhe opções e envia uma solicitação. Eles aparecem em cadastros, pesquisas, login, checkout, filtros e inscrições. Por isso, precisam ser claros, previsíveis, validados corretamente e fáceis de usar por mouse, teclado e leitores de tela.



The image shows a web browser window with the address bar displaying "/cadastro.html". The page content is a registration form titled "Cadastro". It contains three input fields: "Nome" with the value "Maria Silva", "E-mail" with the value "maria@email.com", and "Senha" with masked characters ".....". Below the fields is a blue button labeled "Enviar".

Cadastro

Nome
Maria Silva

E-mail
maria@email.com

Senha
.....

Enviar

A estrutura básica do elemento form

O elemento form delimita o conjunto de controles que será enviado. O atributo action indica o destino dos dados e method define a forma de envio. Mesmo quando a aula usa apenas HTML local, é importante entender essa estrutura porque ela prepara a comunicação com servidores e aplicações reais.

```
<form action="/cadastro" method="post">  
  <label for="nome">Nome</label>  
  <input id="nome" name="nome">  
  <button type="submit">Enviar</button>  
</form>
```

Rótulos e nomes dos campos

Cada campo deve ter um rótulo visível e associado ao controle. O atributo for do label deve apontar para o id do input. Já o atributo name é o nome enviado junto com o valor. Sem esses detalhes, o formulário pode parecer correto, mas fica frágil para servidores e tecnologias assistivas.

```
<label for="email">E-mail</label>  
<input id="email" name="email"  
      type="email">
```

Números, datas e limites

Campos como number, date, time, month e range ajudam o navegador a oferecer controles mais adequados ao tipo de dado. Atributos como min, max e step orientam limites e intervalos. Essa combinação reduz digitação manual, melhora a consistência dos dados e facilita a validação inicial.

```
<input type="number" min="1" max="10">  
<input type="date" name="nascimento">  
<input type="range" min="0" max="100">
```

Escolhas: radio, checkbox e select

Quando o usuário precisa escolher, devemos selecionar o controle conforme a regra da pergunta. Radio é adequado para uma única opção entre alternativas. Checkbox funciona para itens independentes ou múltiplas escolhas. Select economiza espaço quando há muitas opções, mas pode esconder alternativas importantes.

```
<input type="radio" name="turno"> Manhã  
<input type="checkbox" name="aceite"> Aceito  
<select name="curso">  
  <option>ADS</option>  
</select>
```

Área de texto e upload

Textarea é usado quando o usuário precisa digitar textos maiores, como observações, justificativas ou mensagens. O input file permite selecionar arquivos do dispositivo. Nesses casos, instruções claras são indispensáveis: limite de tamanho, formatos aceitos, exemplo esperado e informação sobre obrigatoriedade.

```
<textarea name="mensagem"
rows="5"></textarea>
<input type="file" accept=".pdf, .docx">
```

Botões e intenção de ação

Botões precisam deixar clara a ação que será executada. O tipo submit envia o formulário; reset limpa valores; button não executa envio por padrão e costuma ser usado com JavaScript. Em interfaces reais, o texto do botão deve ser específico, como “Salvar inscrição”, em vez de apenas “OK”.

```
<button type="submit">Salvar</button>  
<button type="reset">Limpar</button>  
<button type="button">Pré-visualizar</button>
```

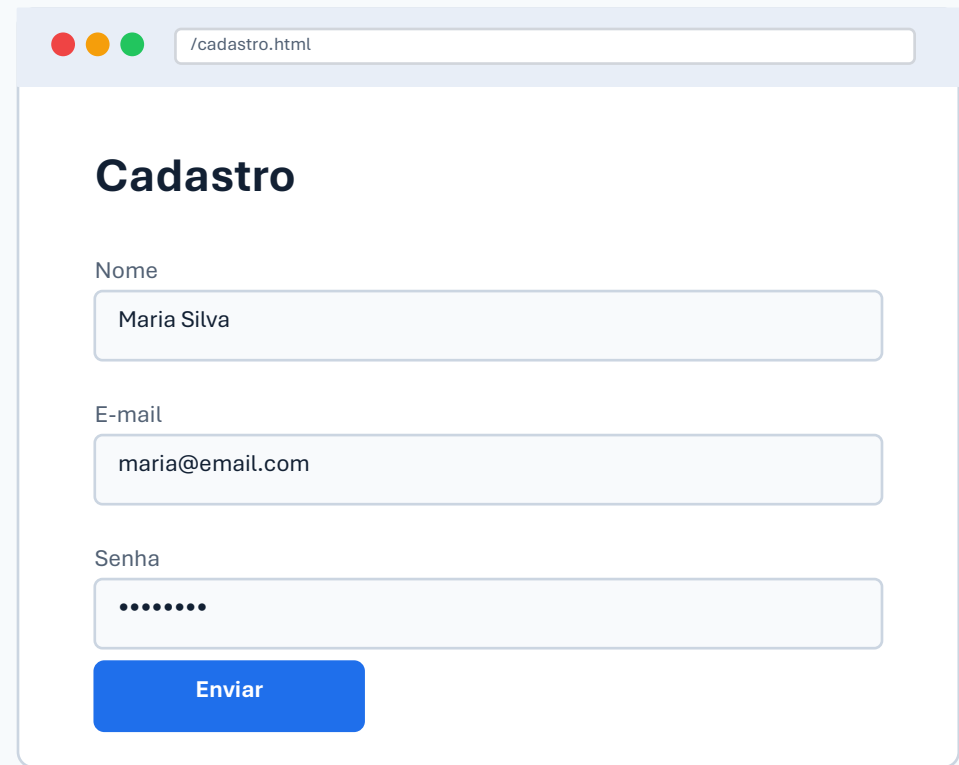
Validação nativa do HTML

A validação nativa do HTML permite indicar regras simples sem escrever JavaScript. O navegador pode impedir o envio quando `required`, `minlength`, `maxlength`, `min`, `max`, `type` e `pattern` não são respeitados. Essa validação melhora a experiência, mas não substitui a verificação de segurança no servidor.

```
<input type="email" required>  
<input minlength="8" maxlength="20">  
<input pattern="[0-9]{5}">
```

required, minlength e maxlength

O atributo `required` informa que o campo precisa ser preenchido antes do envio. `Minlength` e `maxlength` limitam a quantidade de caracteres, ajudando a padronizar entradas como nomes de usuário, senhas e códigos. Essas regras devem fazer sentido para o usuário e ser acompanhadas de instruções visíveis.



The image shows a web browser window with the address bar displaying `/cadastro.html`. The page title is **Cadastro**. It contains three input fields: 'Nome' with the value 'Maria Silva', 'E-mail' with the value 'maria@email.com', and 'Senha' with masked characters '.....'. A blue 'Enviar' button is located below the password field.

Cadastro

Nome
Maria Silva

E-mail
maria@email.com

Senha
.....

Enviar

pattern e expressões regulares

O atributo pattern permite exigir que o valor siga uma expressão regular. Ele é útil para códigos, matrículas e formatos simples, mas deve ser usado com cuidado. Máscaras muito rígidas podem rejeitar entradas válidas. Sempre informe o formato esperado por texto, não apenas por placeholder.

```
<label for="cep">CEP</label>  
<input id="cep" name="cep"  
       pattern="[0-9]{5}-[0-9]{3}"  
       placeholder="29000-000">
```

Mensagens de erro compreensíveis

Uma boa validação não apenas bloqueia o envio; ela ajuda o usuário a corrigir o problema. Mensagens como “inválido” são vagas. Prefira explicar o que está errado e como ajustar. Para acessibilidade, o erro precisa estar em texto, próximo ao campo e perceptível por tecnologias assistivas.

```
Campo: E-mail  
Valor: maria@  
Erro ruim: inválido  
Erro bom: informe um e-mail completo, como  
nome@dominio.com
```

Acessibilidade desde o primeiro campo

Acessibilidade significa permitir que pessoas com diferentes capacidades, dispositivos e contextos consigam usar a página. Em formulários, isso envolve rótulos associados, ordem lógica de navegação, foco visível, mensagens de erro claras, contraste adequado e possibilidade de preencher tudo usando apenas o teclado.

Teste sem mouse



A ordem visual deve acompanhar a ordem de foco.

label, fieldset e legend

O label identifica um campo individual, enquanto fieldset e legend agrupam controles relacionados. Essa organização é muito importante em listas de radio e checkbox, porque o usuário precisa entender a pergunta geral e cada alternativa. Para leitores de tela, o grupo fornece contexto antes da escolha.

```
<fieldset>
  <legend>Turno preferido</legend>
  <label><input type="radio" name="turno">
Manhã</label>
  <label><input type="radio" name="turno">
Noite</label>
</fieldset>
```

Ajuda contextual com aria-describedby

Quando um campo precisa de explicação adicional, podemos associar um texto de ajuda com `aria-describedby`. Isso conecta o input a uma descrição complementar, útil para instruções, dicas e erros. O recurso deve complementar HTML semântico, não substituir labels visíveis e bem escritos.

```
<label for="senha">Senha</label>
<input id="senha" aria-describedby="ajuda-
senha">
<small id="ajuda-senha">
  Use pelo menos 8 caracteres.
</small>
```

Teclado, foco e ordem de leitura

Muitos usuários navegam com Tab, Shift+Tab, Enter, Espaço e setas. Portanto, a página precisa ter ordem lógica de foco e indicação visual clara do elemento ativo. Evite remover outline sem alternativa. Um bom formulário deve ser completamente compreensível e operável sem uso do mouse.

Teste sem mouse



A ordem visual deve acompanhar a ordem de foco.

CSS: separação entre conteúdo e aparência

CSS significa Cascading Style Sheets, ou folhas de estilo em cascata. Ele descreve como elementos HTML são apresentados: cores, espaçamentos, fontes, bordas, alinhamento e layout. A grande vantagem é separar conteúdo e formatação, permitindo reutilizar regras e alterar várias páginas a partir de um único arquivo.

```
p { color : blue ; }
```

seletor propriedade valor

regra CSS = seletor + declarações

Três formas de incluir CSS

O CSS pode aparecer inline, dentro do atributo style; interno, dentro da tag style no head; ou externo, em um arquivo .css referenciado por link. Para projetos reais, a folha externa costuma ser preferida, pois facilita reaproveitamento, organização e manutenção entre várias páginas.

```
<!-- Inline -->
<p style="color: blue">Texto</p>

<!-- Interno -->
<style>p { color: blue; }</style>

<!-- Externo -->
<link rel="stylesheet" href="style.css">
```

Sintaxe: seletor, propriedade e valor

Uma regra CSS normalmente possui um seletor e um bloco de declarações. O seletor indica quais elementos serão afetados. Cada declaração possui propriedade e valor, separados por dois pontos e finalizados por ponto e vírgula. Essa estrutura simples sustenta desde estilos básicos até interfaces completas.

```
p { color : blue ; }
```

seletor propriedade valor

regra CSS = seletor + declarações

Seletores básicos: elemento, classe e id

O seletor de elemento aplica estilo a uma tag, como `p` ou `h1`. A classe, indicada por ponto, pode ser reutilizada em vários elementos. O id, indicado por cerquilha, identifica um elemento específico. Em CSS moderno, prefira classes para padrões reutilizáveis e reserve id para casos realmente únicos.

```
p { color: #333; }  
.destaque { font-weight: bold; }  
#principal { max-width: 960px; }
```

Combinadores: descendente e filho

Seletores podem considerar a relação entre elementos. O descendente usa espaço, como `form input`, atingindo inputs em qualquer nível dentro do form. O filho direto usa `>`, como `form > input`, alcançando apenas elementos imediatamente dentro do pai. Isso ajuda a aplicar estilos com mais precisão.

`html`

raiz do documento

`body`

conteúdo visível

`form`

grupo de campos

`label / input / button`

filhos e descendentes

Exemplo: `form input` ≠ `form > input`

Pseudo-classes úteis em formulários

Pseudo-classes representam estados dos elementos. Em formulários, `:focus` indica o campo ativo, `:invalid` sinaliza valor inválido, `:required` seleciona campos obrigatórios e `:hover` mostra interação do ponteiro. Elas permitem feedback visual imediato, mas não devem depender apenas de cor para transmitir informação.

```
input:focus { outline: 3px solid #2563eb; }  
input:invalid { border-color: #dc2626; }  
button:hover { filter: brightness(.95); }
```

Cascata: como o navegador decide

Quando várias regras CSS afetam o mesmo elemento, o navegador precisa decidir qual valor será aplicado. A cascata considera origem da regra, importância, especificidade e ordem de aparecimento. Entender essa decisão evita “brigas” no CSS e ajuda a corrigir estilos sem recorrer imediatamente a `!important`.

1. origem navegador, usuário, autor

2. importância `!important`

3. especificidade id > classe > elemento

4. ordem a última regra vence

Herança: estilos que passam adiante

Algumas propriedades CSS são herdadas pelos elementos filhos, como color e font-family. Outras, como margin, border e padding, normalmente não são herdadas. Saber o que herda evita repetição desnecessária e ajuda a definir estilos gerais no body, enquanto ajustes específicos ficam em componentes menores.

html

raiz do documento

body

conteúdo visível

form

grupo de campos

label / input / button

filhos e descendentes

Exemplo: form input $\neq$ form > input

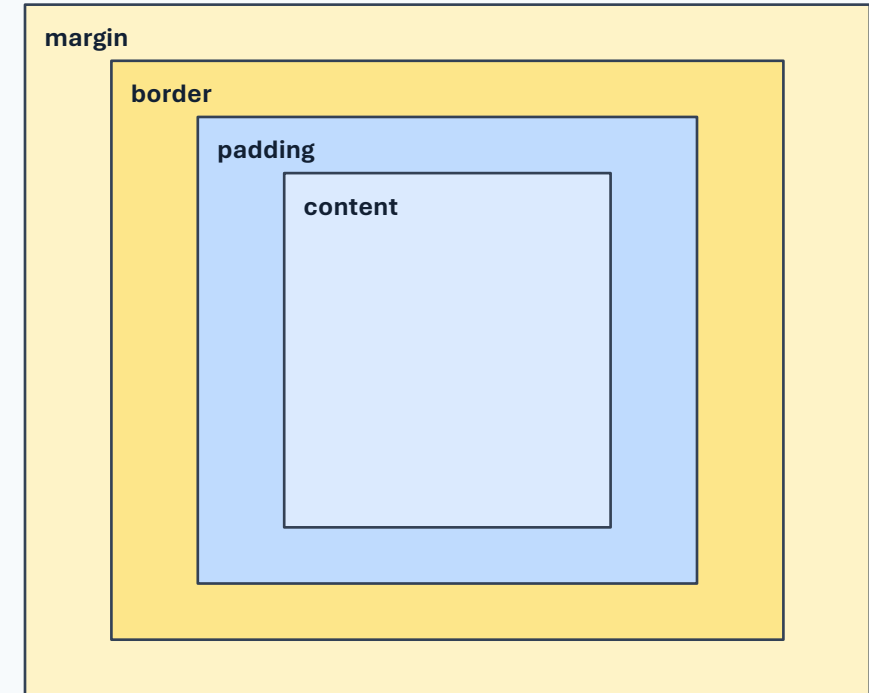
Especificidade em linguagem simples

Especificidade é o peso de um seletor na disputa da cascata. Ids pesam mais que classes, classes pesam mais que elementos. Se duas regras têm a mesma força e origem, geralmente a que aparece depois vence. Manter seletores simples deixa o CSS mais previsível e fácil de depurar.

```
#menu      → mais específico  
.ativo     → intermediário  
button    → mais geral  
button.ativo → combinação
```

Modelo de caixas: tudo é uma caixa

No CSS, cada elemento renderizado pode ser entendido como uma caixa formada por conteúdo, padding, border e margin. O conteúdo é a área interna; padding cria espaço dentro da borda; border desenha o contorno; margin cria distância externa. Compreender isso evita problemas de alinhamento e largura inesperada.



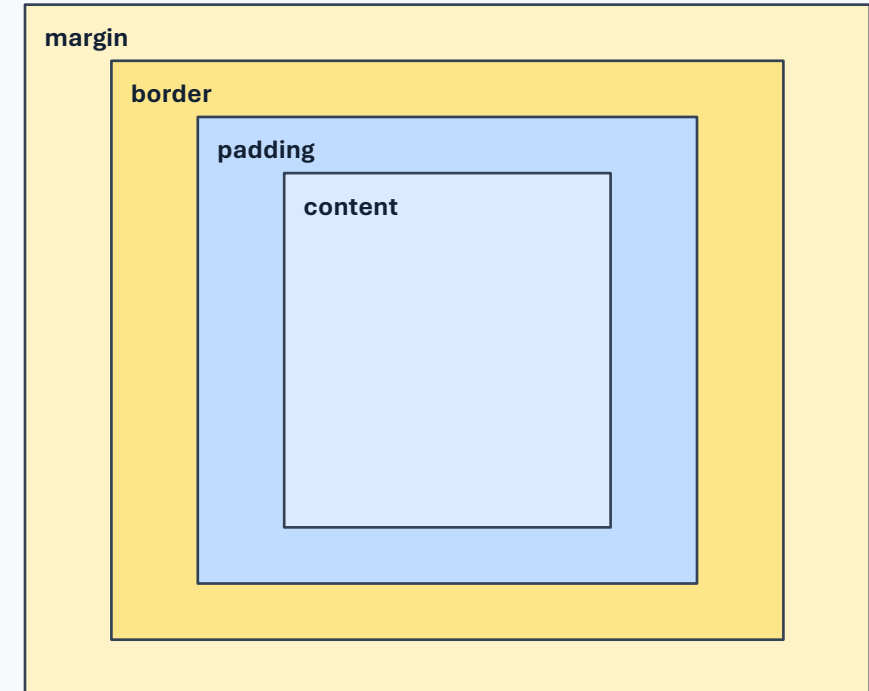
box-sizing e cálculo de largura

Por padrão, a largura definida em CSS se aplica ao conteúdo, e padding e border podem aumentar o tamanho final do elemento. Com `box-sizing: border-box`, `width` passa a incluir conteúdo, padding e border. Essa configuração torna layouts mais previsíveis, especialmente em formulários e componentes responsivos.

```
.card {  
  box-sizing: border-box;  
  width: 300px;  
  padding: 20px;  
  border: 2px solid #ccc;  
}
```

Margem, padding e borda na prática

Margin separa um elemento dos vizinhos. Padding cria respiro entre conteúdo e borda. Border evidencia limites visuais. Em formulários, esses três recursos ajudam a criar campos confortáveis, grupos bem separados e botões legíveis. O excesso, porém, pode quebrar o layout em telas pequenas.



Unidades e responsividade

CSS oferece unidades absolutas e relativas. Pixels são previsíveis, mas rem, em, %, vw e vh se adaptam melhor a contextos variados. Em páginas responsivas, combinamos width, max-width, media queries e unidades relativas para criar interfaces que funcionem em notebooks, tablets e celulares.

```
width: 100%;  
max-width: 480px;  
padding: 1rem;  
font-size: clamp(1rem, 2vw, 1.25rem);
```

Tipografia: leitura antes de decoração

Tipografia envolve fonte, tamanho, peso, espaçamento entre linhas e hierarquia visual. Em páginas web, o objetivo principal é leitura. Uma boa escolha de font-family, font-size e line-height ajuda o usuário a compreender rapidamente títulos, rótulos, instruções, erros e conteúdo principal.

```
body {  
  font-family: Arial, sans-serif;  
  font-size: 16px;  
  line-height: 1.5;  
}  
h1 { font-size: 2rem; }
```

Fonte, peso e espaçamento

Propriedades de texto e fonte controlam a aparência das palavras. Font-weight define intensidade do negrito; font-size define tamanho; line-height melhora respiro vertical; letter-spacing ajusta distância entre letras; text-align organiza alinhamento. Em formulários, rótulos pequenos demais e textos apertados prejudicam a compreensão.

```
font-family  
font-size  
font-weight  
line-height  
letter-spacing  
text-align
```

Cores, contraste e significado

Cores ajudam a criar identidade visual e indicar estados, mas não devem ser o único meio de comunicação. Um campo com erro precisa de texto explicativo, não apenas borda vermelha. Também é necessário garantir contraste suficiente entre texto e fundo, principalmente em rótulos, botões e mensagens.

```
:focus → borda azul + outline  
erro    → texto + ícone + borda  
sucesso → mensagem clara  
contraste adequado
```

Prática 1: criar um formulário acessível

Para exercitar, crie um arquivo `index.html` com um formulário de inscrição simples. Inclua nome, e-mail, curso, turno preferido e aceite dos termos. Associe labels corretamente, use `required` nos campos obrigatórios, agrupe radios com `fieldset` e teste se tudo pode ser preenchido usando apenas o teclado.

1. Criar `index.html`
2. Adicionar `form`, `label` e `input`
3. Usar `required` e `type="email"`
4. Agrupar opções com `fieldset`
5. Testar envio e navegação por Tab

Prática 2: estilizar com CSS externo

Depois do HTML, crie um arquivo `style.css` e conecte-o no head com `link`. Defina fonte geral, largura máxima do formulário, espaçamento entre grupos, aparência dos inputs e estilo do botão. Acrescente regras para `:focus` e `:invalid`, observando como o CSS melhora a leitura sem alterar o conteúdo.

1. Criar `style.css`
2. Referenciar com `<link>`
3. Estilizar `body` e `form`
4. Aplicar classes em grupos
5. Criar `:focus` e `:invalid`

Checklist rápido de acessibilidade

Antes de considerar um formulário pronto, revise alguns pontos: todos os campos têm label; instruções aparecem antes do erro; mensagens são claras; o foco é visível; a ordem de tabulação faz sentido; radio e checkbox estão agrupados; contraste e tamanho de texto permitem leitura confortável.

- ✓ label associado
- ✓ instruções visíveis
- ✓ erro textual
- ✓ foco visível
- ✓ navegação por teclado
- ✓ contraste suficiente

Prática 3: ver uma página sem CSS

Uma atividade interessante é desabilitar o CSS de um site conhecido usando a extensão Web Developer. Ao remover os estilos, fica mais fácil perceber a diferença entre estrutura e apresentação. Sites bem construídos ainda mantêm leitura básica, ordem lógica de conteúdo e elementos compreensíveis.

1. Instalar Web Developer
2. Abrir um site conhecido
3. Menu CSS → Disable All Styles
4. Comparar antes/depois
5. Identificar estrutura HTML restante

Erros comuns a evitar

Alguns erros aparecem com frequência em páginas iniciais: usar placeholder como único rótulo, esquecer name nos campos, remover outline, criar mensagens de erro apenas por cor, repetir CSS inline em cada elemento, abusar de id e !important ou usar espaçamentos que funcionam apenas em uma resolução.

```
Sem label  
Placeholder como único rótulo  
CSS inline em excesso  
!important em tudo  
Outline removido  
Contraste fraco
```

Exercício de fixação

Como atividade de fixação, desenvolva um formulário de inscrição para um minicurso. Ele deve conter dados pessoais, escolha de turno, justificativa em textarea e botão de envio. O CSS deve ficar em arquivo externo. Ao final, teste validações, navegação por teclado e contraste visual.

Desafio:

Criar um formulário de inscrição

Estilizar com CSS externo

Testar validação nativa

Registrar 3 melhorias de acessibilidade

Referências e materiais de apoio

1. Material anexo: Desenvolvedor Web — IFB, seção Fundamentos de Webdesign, HTML e Introdução ao CSS.
2. Material anexo: Assunto 5 — CSS, formatação com folhas de estilo, seletores, margens, bordas e alinhamento.
3. Material anexo: Introdução ao CSS — roteiro da extensão Web Developer para desabilitar CSS.
4. MDN Web Docs — HTML form, input, label, fieldset, validação de formulários e CSS.
5. W3C — Web Content Accessibility Guidelines (WCAG) 2.2 e Quick Reference.
6. web.dev — Learn Forms, Learn Accessibility e Learn CSS Box Model.
7. Visual Studio Code — editor gratuito; W3C Markup Validation Service — validação de marcação HTML.
8. Sugestões de vídeos: canais Curso em Vídeo, MDN/Web Docs, Kevin Powell e Google Chrome Developers sobre HTML, CSS e acessibilidade.